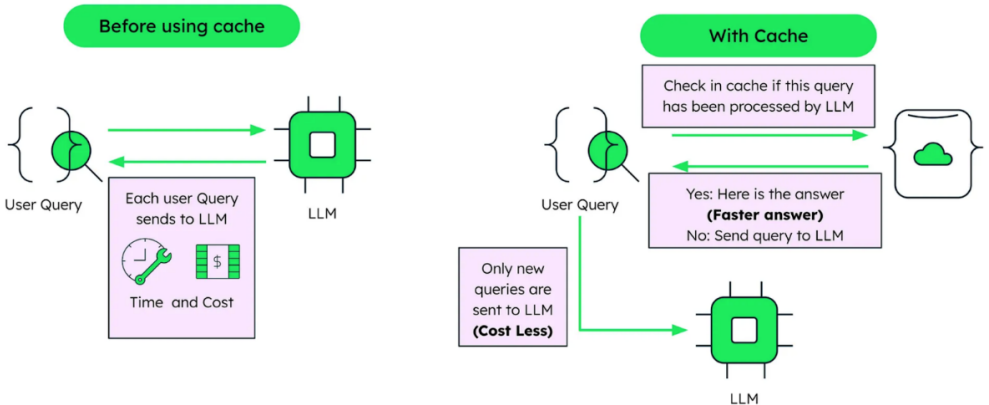


Build a Semantic Caching Layer

Catch repeated questions before they ever reach your LLM:
even when the wording is different.



Exact-Match Caching Misses **Almost Every Repeat**

A standard cache (`dict` , Redis) only hits if the incoming request is byte-for-byte identical to a previous one. But real users rarely phrase things the same way twice:

- 💬 "What's your refund policy?"
- 💬 "How do refunds work?"
- 💬 "Can I get my money back?"

All three mean the same thing, but an exact-match cache treats them as three separate, uncached LLM calls. **Semantic caching** compares the **meaning** (via embeddings) instead of the exact string, so all three can hit the same cached answer.

✖ EXACT-MATCH CACHE

- "refund policy?": MISS
- "how refunds work?": MISS
- "get money back?": MISS

3 expensive LLM calls

✔ SEMANTIC CACHE

- "refund policy?": MISS (Stores)
- ✔ "how refunds work?": HIT
- ✔ "get money back?": HIT

1 LLM call, 2 instant hits!



A Small, High-Leverage Project **Most Beginners Skip**

Understanding what happens under the hood turns a black box into a foundational engineering superpower.

- 📈 **Highest ROI cost optimization:** It is one of the highest ROI cost-optimization techniques for any LLM app with repeated or similar user queries (FAQ bots, support assistants, internal tools).
- 🎓 **Genuinely foundational skills:** It teaches you embeddings, vector similarity, and cache design all in one project: foundational concepts across modern AI engineering.
- ⚡ **Mastering the tradeoffs:** Existing tools (like GPTCache) exist, but building your own first means you understand exactly what threshold, storage, and invalidation tradeoffs are happening under the hood.

👤 Skill Convergence Architecture



Embeddings



Vector Similarity



Cache Design



Semantic Caching Layer (Your Hand-Rolled Engine)



What We Are Building, **End to End**

Here is the complete workflow of a semantic caching pipeline from incoming query to answer delivery:



```
>_ dependencies.sh
```

```
uv add sentence-transformers faiss-cpu numpy
```



Roadmap: We will build each component step by step, starting with the embedding model and ending with a FAISS-powered vector store.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

Turning the Question into a Comparable Vector

Converts every incoming query into a numerical embedding: the exact same representation used to compare against cached queries.

embed_query.py

```
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer("BAAI/bge-small-en-v1.5")

def embed_query(query: str):
    return embedder.encode(query, normalize_embeddings=True)

query_vector = embed_query("What's your refund policy?")
```

"What's your refund policy?"



Embedding Model (BAAI/bge-small-en-v1.5)



[0.021, -0.114, 0.087, ...] <- 384-dimensional vector

CONSISTENCY RULE

Use the same embedding model consistently for every cached query and every new incoming query: mixing models makes similarity scores meaningless, placing text in completely different vector spaces.



Cache Storage Structure: What Needs Storing

Defines the data structure for each cached entry: you need the original query, its embedding, and the LLM's answer.

cache_store.py

```
from dataclasses import dataclass
import numpy as np

@dataclass
class CacheEntry:
    query: str
    embedding: np.ndarray
    answer: str
    timestamp: float

cache_store: list[CacheEntry] = [] # simple in-memory version to start
```

📋 CACHEENTRY RECORD ANATOMY

```
query: "What's your refund policy?"
embedding: [0.021, -0.114, 0.087, ...]
answer: "Refunds are accepted within 30 days..."
timestamp: 1721558400.0
```

Why start simple? Starting with a plain Python list is intentional: get the logic right first with something simple and inspectable, then swap in FAISS (Slide 15) once you need to search efficiently over thousands of entries.



Checking the Cache: **Similarity Search**

Computes cosine similarity between the incoming query's embedding and every cached entry's embedding, returning the closest match.

similarity_search.py

```
import numpy as np

def find_closest_cache_entry(query_vector, cache_store:
list[CacheEntry]):
    if not cache_store:
        return None, 0.0

    similarities = [
        np.dot(query_vector, entry.embedding) for entry in
cache_store
    ]
    best_idx = int(np.argmax(similarities))
    return cache_store[best_idx], similarities[best_idx]

closest_entry, score = find_closest_cache_entry(query_vector,
cache_store)
```

New Query Vector ↓

Cached #1

sim: 0.42



Cached #2

sim: 0.91

Cached #3

sim: 0.68

↑ **Highest similarity: this candidate wins and undergoes threshold verification**



Math Note: Because embeddings are normalized (Slide 5), the dot product directly gives cosine similarity: no extra division needed!



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 **Repost**

The Similarity Threshold: **Similar Enough?**

Compares the best match's similarity score against a chosen threshold: above it, return the cached answer; below it, treat as a cache miss.

check_cache.py

```
SIMILARITY_THRESHOLD = 0.90 # tune this based on your own testing

def check_cache(query: str, cache_store: list[CacheEntry]):
    query_vector = embed_query(query)
    closest_entry, score = find_closest_cache_entry(query_vector,
    cache_store)

    if closest_entry and score ≥ SIMILARITY_THRESHOLD:
        return closest_entry.answer # cache HIT
    return None # cache MISS
```

☞ THRESHOLD DECISION NUMBER LINE

0.0 (Different)

Threshold = 0.90

1.0 (Identical)

× MISS Zone (Call LLM)

✓ HIT Zone (Return Cache)

This threshold is the single most important tuning parameter in the whole system: too low, and you return wrong answers to genuinely different questions; too high, and you rarely get cache hits at all.



Cache Hit Path: **Instant Response**

Skips the LLM call entirely and returns the cached answer immediately: this is where actual cost and latency savings come from.

get_answer.py

```
def get_answer(query: str, cache_store: list[CacheEntry], llm):
    cached_answer = check_cache(query, cache_store)
    if cached_answer is not None:
        print("Cache HIT - no LLM call made")
        return cached_answer

    # cache miss path: covered next slide
    return handle_cache_miss(query, cache_store, llm)
```

✓ **CACHE HIT FLOW** (`SIM >= 0.90`)

Query



Embed



Search



**Return
Cached
Answer**

(No LLM call,
~instant)



Performance Impact: Drops response latency from ~2,000ms down to ~15ms while completely zeroing out API token bills for repeat intents.



Cache Miss Path: Store & Compound

Falls through to an actual LLM call, then stores the new query, answer, and embedding as a fresh cache entry for future hits.

handle_cache_miss.py

```
def handle_cache_miss(query: str, cache_store: list[CacheEntry], llm):  
    print("Cache MISS - calling the LLM")  
    answer = llm.invoke(query).content  
  
    new_entry = CacheEntry(  
        query=query,  
        embedding=embed_query(query),  
        answer=answer,  
        timestamp=time.time()  
    )  
    cache_store.append(new_entry)  
    return answer
```

🔗 CACHE MISS FLOW ($SIM < 0.90$)

Search Miss



Call LLM



Store (query,
embedding, answer)
+1 Cache Entry added

Every miss makes your cache slightly smarter for next time: this is what makes semantic caching compound in value the longer a system runs and the more repeat traffic it sees.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

Repost

Choosing the Threshold: Not a Guess

Build a small evaluation set of query pairs labeled "should hit" or "should miss," then sweep threshold values to find what empirically maximizes accuracy.

evaluate_threshold.py

```
test_pairs = [
    ("What's your refund policy?", "How do refunds work?", True), # should
    hit
    ("What's your refund policy?", "What's your shipping time?", False), #
    should miss
]

def evaluate_threshold(threshold, test_pairs, embedder):
    correct = 0
    for query_a, query_b, should_match in test_pairs:
        sim = np.dot(embed_query(query_a), embed_query(query_b))
        predicted_match = sim >= threshold
        correct += (predicted_match == should_match)
    return correct / len(test_pairs)

for t in [0.80, 0.85, 0.90, 0.95]:
    print(t, evaluate_threshold(t, test_pairs, embedder))
```

☰ THRESHOLD SWEEP EVALUATION RESULTS



Never pick based on gut feeling: build even a small 30 to 50 pair labeled test set from real domain queries and pick the winner empirically.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

🔄 Repost

Cache Invalidation: Time-Based (TTL)

Cached answers can go stale. Adds a time-to-live to each entry so answers based on outdated information automatically expire.

ttl_check.py

```
import time

TTL_SECONDS = 60 * 60 * 24 # 24 hours

def check_cache_with_ttl(query, cache_store, threshold=0.90):
    query_vector = embed_query(query)
    closest_entry, score = find_closest_cache_entry(query_vector, cache_store)

    if closest_entry and score >= threshold:
        age = time.time() - closest_entry.timestamp
        if age < TTL_SECONDS:
            return closest_entry.answer
    return None # expired or no match: treat as miss
```

TTL LIFECYCLE TIMELINE

Entry Created (age = 0s)

Valid Hit Window

TTL Expired (> 24h)

Served directly from cache

Now

Treated as Miss (Refetch)

The right TTL depends entirely on how often your underlying data changes: a static FAQ might tolerate a week-long TTL, while pricing or inventory data might need minutes, not hours.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

Invalidating by **Content & KB Version**

TTL handles time decay, but what when a specific answer instantly becomes wrong because someone updated the underlying policy or database?

- 🔑 **Tag entries by source/topic:** When a document updates, invalidate only entries tagged with that source instead of clearing the whole cache.
- 🔑 **Version your knowledge base:** Increment a version number whenever underlying data changes, invalidating any entry created under older versions.

kb_version_check.py

```
CURRENT_KB_VERSION = 3

@dataclass
class CacheEntry:
    query: str
    embedding: np.ndarray
    answer: str
    timestamp: float
    kb_version: int # tag with the knowledge base version at creation time

def check_cache_versioned(query, cache_store, threshold=0.90):
    closest_entry, score = find_closest_cache_entry(embed_query(query),
    cache_store)
    if closest_entry and score ≥ threshold and closest_entry.kb_version ==
    CURRENT_KB_VERSION:
        return closest_entry.answer
    return None
```

✖ Tagged KB v2 (Old)

~~Answer generated under old policy version~~

Rejected on read (Invalid)

✔ Tagged KB v3 (Current)

Answer matches current knowledge base

Served immediately (Valid Hit)



Full Implementation: Every Piece Assembled

Combining embedding, similarity checking, TTL expiration, and miss-handling into one clean, reusable class:

```
semantic_cache.py

class SemanticCache:
    def __init__(self, embedder, threshold=0.90, ttl_seconds=86400):
        self.embedder = embedder
        self.threshold = threshold
        self.ttl_seconds = ttl_seconds
        self.entries: list[CacheEntry] = []

    def get(self, query: str):
        query_vector = self.embedder.encode(query, normalize_embeddings=True)
        if not self.entries:
            return None

        sims = [np.dot(query_vector, e.embedding) for e in self.entries]
        best_idx = int(np.argmax(sims))
        best_entry, best_score = self.entries[best_idx], sims[best_idx]

        if best_score >= self.threshold and (time.time() - best_entry.timestamp) <
self.ttl_seconds:
            return best_entry.answer
        return None

    def set(self, query: str, answer: str):
        embedding = self.embedder.encode(query, normalize_embeddings=True)
        self.entries.append(CacheEntry(query, embedding, answer, time.time()))

# Usage
cache = SemanticCache(embedder, threshold=0.90)

def get_answer(query, llm):
    cached = cache.get(query)
```



Ready to Run: This exact class handles embedding, cosine dot-product search, threshold verification, and TTL checks in under 30 lines of clean Python.



Scaling Up: From Python List to FAISS

The list-based prototype works fine early on, but searching becomes slow once you have thousands of entries. Here is how to scale:

- ⚡ **FAISS:** Swap the list for a FAISS index once you are past a few thousand entries; exact same inner-product math, but orders of magnitude faster search.
- 🗄️ **Redis with Vector Search:** If you need the cache to persist across server restarts and be shared across multiple app instances.
- 📦 **GPTCache:** A purpose-built open-source library implementing this exact pattern, worth adopting once you understand (and outgrow) your hand-rolled engine.

`faiss_migration.py`

```
import faiss

index = faiss.IndexFlatIP(384) # same inner-product search as before, much
faster at scale
index.add(np.array([e.embedding for e in cache_store]))
```

↑ Storage Backend Evolution Roadmap

Python List
Fine < 1,000 entries



FAISS Index
Scale to millions



Redis / GPTCache
Shared & persistent



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 **Repost**

Measuring Savings: Prove the Value

Track hit rate and estimated cost savings so you can show concretely how much the semantic cache saves in dollars and latency.

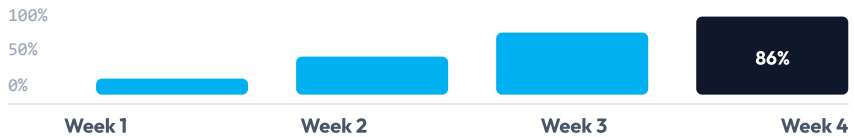
cache_metrics.py

```
class CacheMetrics:
    def __init__(self):
        self.hits = 0
        self.misses = 0

    def record(self, was_hit: bool):
        self.hits += was_hit
        self.misses += not was_hit

    def report(self, cost_per_call=0.002):
        total = self.hits + self.misses
        hit_rate = self.hits / total if total else 0
        savings = self.hits * cost_per_call
        print(f"Hit rate: {hit_rate:.1%} | Estimated savings: ${savings:.2f}")
```

📈 HIT RATE CURVE OVER TIME (CACHE WARMING)



Hit rate climbs naturally as the cache accumulates more user phrasing variations!



Compounding ROI: On an app with 50,000 daily queries and a 60% hit rate, semantic caching saves 30,000 LLM calls per day!



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

🔄 Repost

Handling Multi-Turn Conversations

A standalone question ("What's the refund policy?") caches cleanly. But in multi-turn chat, prior context changes the meaning of follow-ups.

- ✔ **Simplest fix:** Only cache single-turn, context-free queries; skip caching for follow-up questions that reference conversation history.
- ✎ **More advanced fix:** Embed the *resolved* query (after query rewriting resolves pronouns/context, as covered in Agentic RAG) rather than raw follow-up text!

✂ Query Rewriting Pipeline Before Cache Lookup

Turn 1: "What's your refund policy?"

✔ Cacheable directly

Turn 2: "What about for digital products?"

⚠ Ambiguous alone

↓ Step 2.5: Context-Aware Query Rewrite

Resolved: "What is the refund policy for digital products?"

🔍 NOW check cache!



Golden Rule: Never embed unresolved pronouns like "that" or "it": always rewrite the query to be self-contained before generating vectors.



Common Pitfalls

Mistakes that undermine an otherwise-good cache: watch out for these 6 critical errors during production deployment.

- ✖ **No similarity threshold tuning:** Guessing a number instead of testing against labeled query pairs risks returning wrong answers for similar-but-different questions.
- ✖ **No TTL or invalidation strategy:** Serving confidently wrong, outdated answers forever when policies or pricing update.
- ✖ **Caching multi-turn follow-ups without resolving context first:** Ambiguous follow-ups can match the wrong cached entry entirely.
- ✖ **Using a Python list past a few thousand entries:** Search gets slow; migrate to FAISS or Redis before vector scan becomes a bottleneck.
- ✖ **Never measuring hit rate/savings:** You cannot prove (or improve) the value of the cache without tracking hits vs misses.
- ✖ **Mixing embedding models between cache-writes and reads:** Silently breaks all similarity comparisons across the vector space.



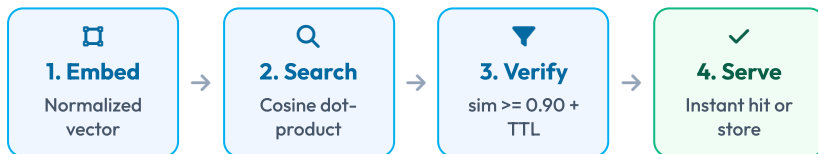
Reliability Checklist: Verify consistent embedding models, tuned thresholds, and active TTL expiration before routing live user traffic.



Cache by Meaning, Not Exact Text 🎯

A semantic cache catches the huge number of "differently worded, same meaning" repeat questions that a normal cache misses entirely: often cutting LLM costs significantly for FAQ-heavy or support-style traffic, once tuned and measured properly.

ARCHITECTURE RECAP (END-TO-END FLOW)



Save this as your semantic caching build guide. Follow for more hands-on AI engineering breakdowns.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost